

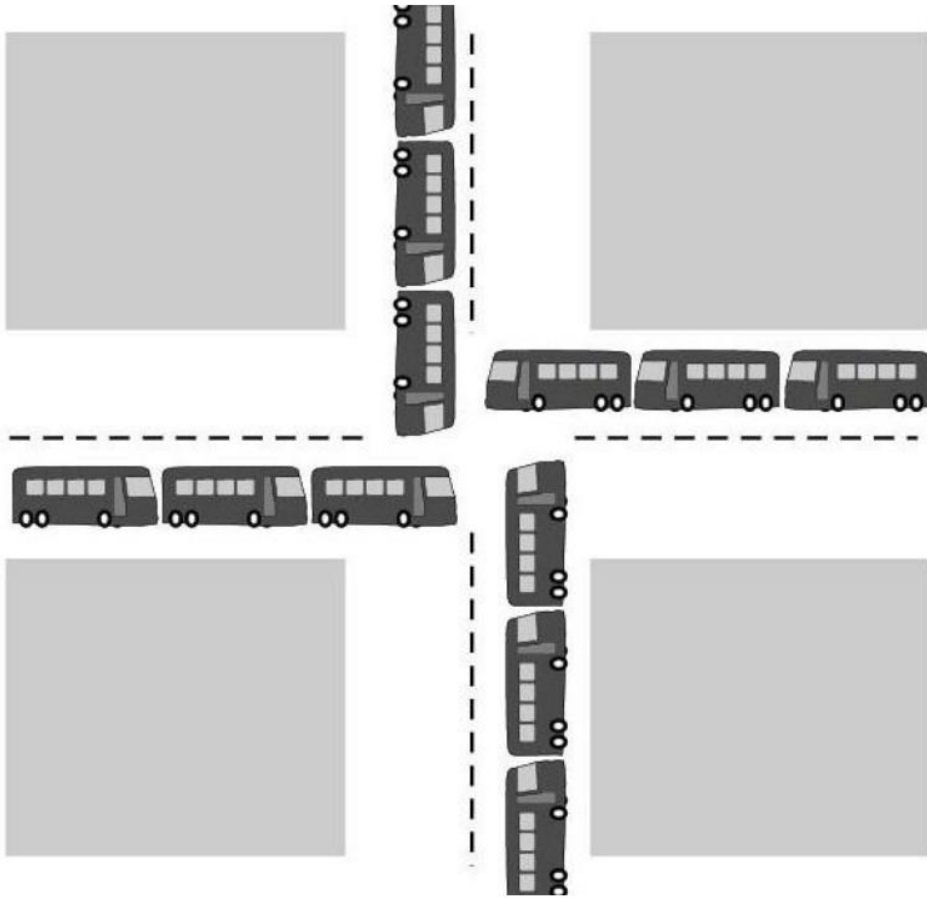
Deadlocks

Unit-4

Deadlock

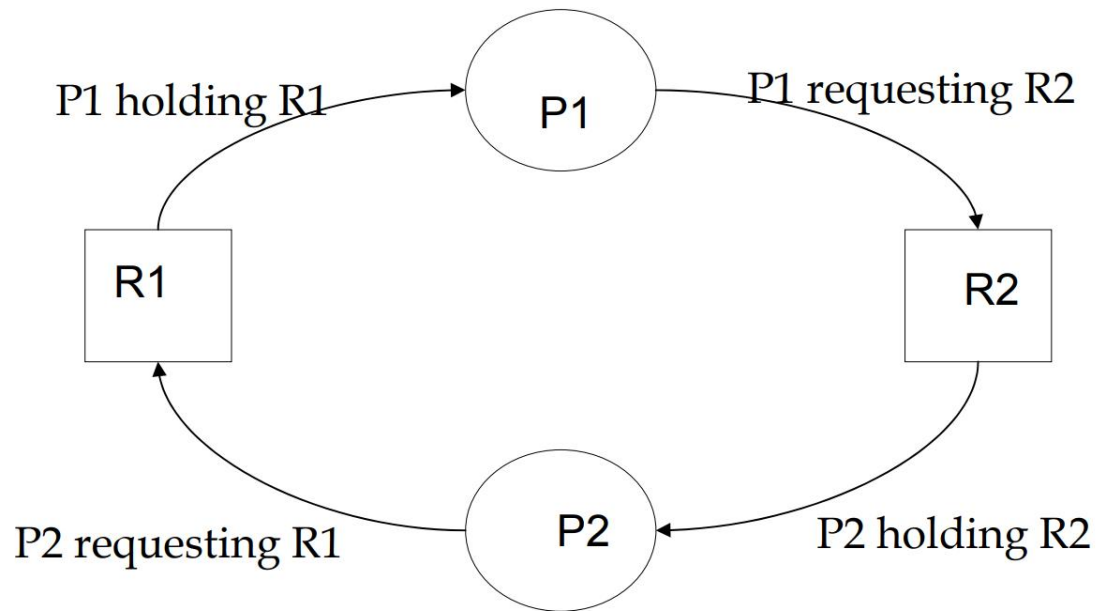
- A process in a multiprogramming system is said to be in deadlock if it is waiting for a particular event that will never occur.
- It is a situation in which, neither the processing task of CPU is completed nor the CPU is released to execute other task.
- Deadlock is a situation where a set of processes are blocked because each process is holding a resource and waiting for another resource acquired by some other process.

Example: A traffic deadlock



- All automobiles trying to cross.
- Traffic Completely stopped.
- Not possible without backing some.

Example: Circular wait deadlock



- Process P1 holds resource R1 and needs resource R2 to continue
- Process P2 holds resource R2 and needs resource R1 to continue
- Result deadlock

System Model

- A process request a resource before using it, and release after using it.
- Steps:
 1. Request the resource.
 2. Use the resource.
 3. Release the resource.
- If the resource is not available when it is requested, the requesting process is force to wait.
- Most deadlocks in OS are developed because of the normal contention for dedicated resources.

System Resources: Preemptable and Non-preemptable resource

- A preemptable resource is the one that can be taken away from the process with no ill (negative) effects. Example: Memory.
- A non-preemptable resources is one that cannot be taken away from process without causing ill (negative) effects. Example: CD resources.
- In case of preemptable resources, reallocating resources can resolve deadlocks.
- Deadlocks that involve non-preemptable resources are difficult to deal with.

Conditions for resource deadlock

Condition 1: Mutual Exclusion condition

Process claims exclusive control of resources they require. Only one process at a time claims exclusive control of the resource. If another process requests that resource, the requesting process must be delayed until the resource has been released.

Condition 2: Hold and Wait

Processes hold resources already allocated to them while waiting for additional resources that are currently being held by another process.

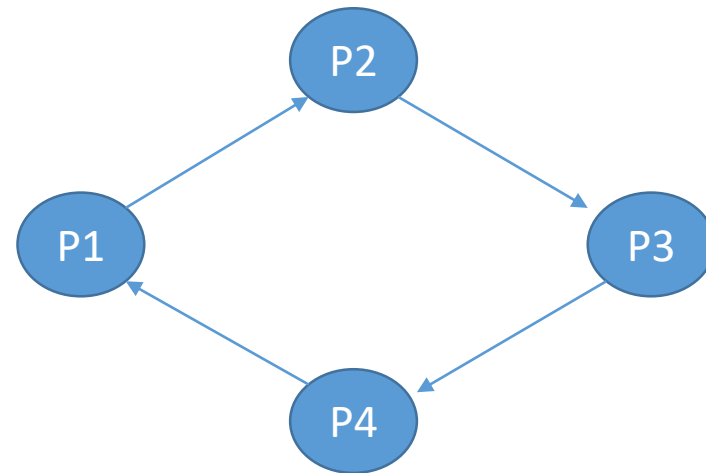
Condition 3: No preemption

Resources previously granted cannot be forcibly taken away from the process. Resources cannot be removed until completed or released voluntarily by the process holding it.

Conditions for resource deadlock (contd..)

Condition 4: Circular wait

Each process holds one or more resources that are requested by next process in the chain. All the processes wait for the resources in a cyclic manner, where the last process waits for the resource held by the first process.



OSTRICH Algorithm

Idea behind OSTRICH Algorithm

There is no good way of dealing with deadlock. So, Ignore the problem altogether.

For most operating systems, deadlock is a rare occurrence. So, the problem of deadlock is ignored, like an ostrich sticking its head in the sand and hoping the problem will go away.

Deadlock is rare, the cost of prevention or detection is high. So, pretend as if there is no problem.

The UNIX and Windows operating system take this approach.

Methods of Deadlock handling

There are mainly four methods for handling deadlock.

1. Deadlock Prevention
2. Deadlock Avoidance (Banker's Algorithm)
3. Deadlock Detection (Resource Allocation Graph)
4. Recovery from Deadlock

Deadlock Prevention

It is the process of designing the system, where there is no chance of having a deadlock.

We can prevent deadlock by eliminating any of the following conditions:

- Elimination of Mutual Exclusion Condition
- Elimination of Hold and Wait Condition
- Elimination of No-Preemption Condition
- Elimination of Circular Wait Condition

Elimination of Mutual Exclusion Condition

- The mutual exclusion condition must hold for non-sharable resources i.e. several processes cannot simultaneously share a single resource.
- Problem: This condition is difficult to eliminate because some resources like printer are non-shareable.
- Shareable resources like read-only file don't require mutually exclusive access and thus cannot be involved in deadlock.

Elimination of Hold and Wait Condition

- There are two possibilities:
 - First, Process request be granted all of the resources it needs at once, prior to execution.
 - Second, Disallow a process from requesting resources whenever it has previously allocated resources.
- This strategy requires that all of the resources a process will need must be requested at once. The system must grant resources on “all or none” basis.
- If the complete set of resources needed by a process is not currently available, then the process must wait until complete set is available.
- While the process waits, however, it may not hold any resources. Thus, the wait for condition is denied and deadlock simply cannot occur.
- Problems: This strategy can lead to serious waste of resources. It can cause indefinite postponement (starvation). Since all the required resources may not become available at once.

Elimination of No-preemption Condition

- This strategy requires that, when a process that is holding some resources, is denied a request for additional resources. The process must release its held resources and, if necessary, request them again together with additional resources.
- Implementation of this strategy denies no-preemptive condition but with high cost.
- Problems: When a process release resources, the process may lose all its work to that point. The possibility of indefinite postponement (starvation) exists as it repeatedly requests and releases the same resources.

Elimination of Circular Wait Condition

- Circular Wait can be denied by imposing a total ordering on all of the resource types and then forcing all processes to request the resources in order (increasing or decreasing).
- This strategy imposes a total ordering of all resources types, and to require that each process requests resources in order (increasing or decreasing). Example of ordering of resources: 1. Card reader 2. Printer 3. Plotter 4. Tape Drive.
- The strategy is process can request resources whenever they want to, but all requests must be made in numerical order.
- Problem: This strategy can eliminate circular wait condition but it may be impossible to find an ordering that satisfies everyone.

Deadlock Avoidance

- This is the process that anticipates deadlock before it actually occurs. It employs an algorithm to assess the possibility that deadlock could occur and act accordingly.
- It is different from deadlock prevention, which guarantees that deadlock cannot occur. However, deadlock avoidance ensures careful allocation of resources.
- The deadlock avoidance algorithm dynamically examines the **resources allocation state** to ensure that a deadlock never exists.
- There are 3 states of the system:
 - Safe state
 - Unsafe state and
 - Deadlock state

Safe and Unsafe state

- When a system can allocate the resources to the process in such a way so that they still avoid deadlock, then the state is called safe state.
- When there exists a safe sequence, then we can say that the system is in the safe state.
- A sequence is in the safe state only if there exists at least one safe sequence.
- A sequence of process P_1, P_2, P_n is a safe sequence for the current allocation state, if for each P_i the resources request that P_i can still be satisfied by currently available resources.

Safe and Unsafe state (contd...)

- If a safe sequence doesn't exist, then the system is in unsafe state, which may lead to deadlock.
- All safe states are deadlock free, but not all unsafe states lead to deadlocks.

Example 1:

Process	Used	Max	Available Units
A	1	6	2
B	2	5	
C	2	4	
D	4	7	

It's Safe. Safe Sequence: C,D,B,A or C,B,A,D or C,D,A,B

Example 2:

Process	Used	Max	Available Units
A	1	6	1
B	2	5	
C	2	4	
D	4	7	

It's Unsafe.

Banker's Algorithm

- It is a resource allocation and deadlock avoidance algorithm.
- It tests for safety by simulating the allocation of predetermined maximum possible amounts of all resources, and then makes an 's-state' check to test for possible deadlock condition for all other pending activities, before deciding whether allocation should be allowed to continue.
- It is sometimes referred to as the detection algorithm.

Banker's Algorithm (contd..)

Let 'n' be the number of processes in the system and 'm' be the number of resources types.

Available:

- It is an one dimensional array of size 'm' indicating the number of available resources of each size.
- $\text{Available}[j]=k$, means there are 'k' instances of resources of type R_j

Max:

- It is a 2-dimensional array of size $n*m$ that defines the maximum demand of resource for each process in the system.
- $\text{Max}[i,j]=k$, means process P_i may request at most k instances of resource type R_j .

Banker's Algorithm (contd..)

Allocation:

- It is a 2-dimensional array of size $n*m$ that defines the number of resources of each type currently allocated to each process.
- $\text{Allocation}[i,j]=k$, means process P_i is currently allocated k instances of resource type R_j .

Need:

- It is a 2-dimensional array of size $n*m$ that indicates the remaining resource need of each process.
- $\text{Need}[i,j]=k$, means process P_i currently need k instances of resource type R_j for its execution.
- $\text{Need}[i,j]=\text{Max}[i,j]-\text{Allocation}[i,j]$

Banker's resource request Algorithm

Step 1: If request $\leq$ need then go to step 2
else Error

Step 2: if request $\leq$ available then go to step 3
else Wait

Step 3: available = available - request
allocation = allocation + request
need = need - request

Step 4: Check new state is safe or not?

Banker's safety algorithm

Step 1:

if need $\leq$ available then

Execute process

Calculate new available as,

available = available + allocation

Step 2:

Otherwise

Do not execute and go forward

Numerical Problem 1

Assume that there are 5 processes, P0 through P4, and 4 types of resources. At T0, we have the following system state:

	Allocation				Max				available			
	A	B	C	D	A	B	C	D	A	B	C	D
P0	0	1	1	0	0	2	1	0	1	5	2	0
P1	1	2	3	1	1	6	5	2				
P2	1	3	6	5	2	3	6	6				
P3	0	6	3	2	0	6	5	2				
P4	0	0	1	4	0	6	5	6				

- Find the need matrix.
- Is the system in safe state? If yes find safe sequence.

Create the need matrix

	Need			
	A	B	C	D
P0	0	1	0	0
P1	0	4	2	1
P2	1	0	0	1
P3	0	0	2	0
P4	0	6	4	2

$\text{Need}[i,j] = \text{Max}[i,j] - \text{Allocation}[i,j]$

Problem with Banker's Algorithm

- It requires fixed number of resources, some processes dynamically change the number of resources.
- It requires the number of resources in advance. It is very difficult to predict the resources in advance.
- This algorithm predicts all process returns within finite time, but the system does not guarantee it.

Deadlock Detection

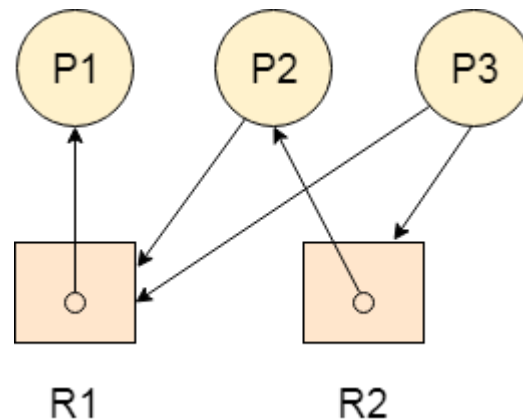
- It is the process of actually determining that a deadlock exists and identifying the processes and resources involved in the deadlock.
- The basic idea is to check allocation against resource availability for all possible allocation sequences to determine if the system is in deadlocked state.
- Once the deadlock is detected, then it should be recovered.

RAG (Resource Allocation Graph)

- The resource allocation graph is the pictorial representation of the state of a system. As its name suggests, the resource allocation graph is the complete information about all the processes which are holding some resources or waiting for some resources.
- It also contains the information about all the instances of all the resources whether they are available or being used by the processes.
- In Resource allocation graph, the process is represented by a Circle while the Resource is represented by a rectangle.

Resource Allocation Graph (Contd..)

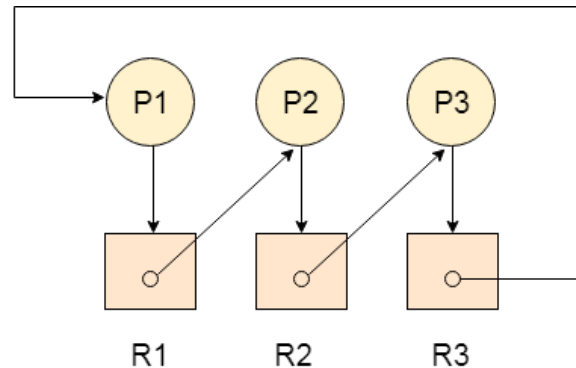
- Let's consider 3 processes P1, P2 and P3, and two types of resources R1 and R2. The resources are having 1 instance each. According to the graph, R1 is being used by P1, P2 is holding R2 and waiting for R1, P3 is waiting for R1 as well as R2.



- The graph is deadlock free since no cycle is being formed in the graph.

Deadlock Detection using RAG

- If a cycle is being formed in a Resource allocation graph where all the resources have the single instance then the system is deadlocked. In Case of Resource allocation graph with multi-instanced resource types, Cycle is a necessary condition of deadlock but not the sufficient condition.
- The following example contains three processes P1, P2, P3 and three resources R1, R2, R3. All the resources are having single instances each.



- If we analyze the graph then we can find out that there is a cycle formed in the graph since the system is satisfying all the four conditions of deadlock.

Recovery from Deadlock

- Real time OS use deadlock recovery. Traditional OS like Windows doesn't deal with deadlock recovery.
- Some common recovery methods are:

For Resource

- a) Preempt the resource – snatch the resource from one process and give it to another process.
- b) Rollback to safe state – the OS can roll back the system to previous safe state.

Recovery from Deadlock (contd...)

For Process

- a) Kill a process – killing a process can solve the problem, but the bigger concern is to determine which process to kill. OS kills a process which has done least amount of work until now.
- b) Kill all process – It is not appropriate, but can be implemented if the problem becomes very serious. Killing all process will result in loosing of all the task in progress.